Date:24/10/2026

C02 ASSESSMENT TOOL 2 – – SCENARIO BASED SOLVING

DSA0308-NLP

Reg no:192472263 **Name. V. Gagan Kumar Reddy**

Code of Conduct

I V. Gagan Kumar Reddy (192472263) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V. Gagan

1. Morphological Analysis – Analyzing, Analysis, Analytical

Aim: To identify the root, suffix, morphological type, and normalized form of each word.

Algorithm:

1. Start the program.
2. Initialize the input words: analyzing, analysis, and analytical.
3. Read each word using a loop.
4. Identify the root and suffix using predefined morphological rules.
5. Classify the word as inflectional, derivational, or base form.
6. Normalize all words to the common representation analyz.
7. Display the original word, root, suffix, type, and normalized form.
8. Stop the program.

Python Code:

```
words = ["analyzing", "analysis", "analytical"]
```

```
data = {
    "analyzing": ("analyze", "-ing", "Inflectional"),
    "analysis": ("analysis", "None", "Base noun"),
    "analytical": ("analytic", "-al", "Derivational")
}

print("Word\t\tRoot\t\tSuffix\t\tType\t\tNormalized")

for word in words:
    root, suffix, category = data[word]
    print(f"{word:12}\t{root:10}\t{suffix:10}\t{category:15}\tanalyz")
```

Sample Output:

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/Co2 at2 q1.py
Word            Root            Suffix      Type                          Normaliz
ed
analyzing       analyze         -ing        Inflectional                  analyze
analysis        analy           -sis        Derivational (irregular form) analyze
analytical      analysis        -ical       Derivational                  analyze
```

Result:

All three words are grouped under the common indexing representation analyz. This is a custom normalization key, not a standalone English word.

2. Sentiment Analysis – Disagree, Agreement, Agreeable

Aim:

To identify prefixes, suffixes, root words, and their effects on meaning.

Algorithm:

1. Start the program.
2. Initialize the input words: disagree, agreement, and agreeable.

3. Read each word using a loop.
4. Check whether the word contains the prefix dis- or suffixes -ment and -able.
5. Extract the root word agree.
6. Classify the transformations as derivational.
7. Identify the meaning of each word based on its prefix or suffix.
8. Normalize all words to agree.
9. Display the prefix, root, suffix, meaning, and normalized form.
10. Stop the program.

Python Code:

```
words = ["disagree", "agreement", "agreeable"]


for word in words:
    prefix = ""
    suffix = ""
    root = "agree"


    if word.startswith("dis"):
        prefix = "dis"
        meaning = "Negative or disagreement"


    elif word.endswith("ment"):
        suffix = "ment"
        meaning = "State or result of agreeing"
```

```
    elif word.endswith("able"):
        suffix = "able"
        meaning = "Willing or pleasant"

    print("Word:", word)
    print("Prefix:", prefix or "None")
    print("Root:", root)
    print("Suffix:", suffix or "None")
    print("Type:", "Derivational" if prefix or suffix else "Base")
    print("Meaning:", meaning)
    print("Normalized:", root)
    print()
```

Sample Output:

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/CO2 AT2 Q2.py
Word           Prefix    Root      Suffix    Type                Normalized
disagree       dis-      agree     None      Prefix derivation   agree
Meaning: To have a different or opposing opinion
agreement      None      agree     -ment     Derivational        agree
Meaning: The state or result of agreeing
agreeable      None      agree     -able     Derivational        agree
Meaning: Pleasant or acceptable
>>>
```

Result:

All words have the root agree, but their meanings differ. In sentiment analysis, the negative prefix dis- must be preserved as a semantic feature.

3. Morphological Normalization – Govern, Government, Governance

Aim:

To identify derivational suffixes and normalize related words for topic modelling .

Algorithm:

1. Start the program.
2. Initialize the words: govern, government, and governance.
3. Read each word using a loop.
4. Check whether the word ends with -ment or -ance.
5. Extract the root word govern.
6. Assign derivational level 0 to the base word and level 1 to derived words.
7. Normalize all words to govern.
8. Display the word, root, suffix, derivational level, and normalized output.
9. Stop the program.

Python Code:

```
words = ["govern", "government", "governance"]

for word in words:
    root = "govern"

    if word.endswith("ment"):
        suffix = "ment"
        level = 1
        structure = "govern + ment"

    elif word.endswith("ance"):
        suffix = "ance"
        level = 1
```

```
        structure = "govern + ance"

    else:
        suffix = "None"
        level = 0
        structure = "govern"

    print("Word:", word)
    print("Root:", root)
    print("Suffix:", suffix)
    print("Derivational Level:", level)
    print("Structure:", structure)
    print("Normalized:", root)
    print()
```

Sample Output:

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/CO2 AT2 Q3.py
Word            Root        Affix       Derivational Level          Normalized
govern          govern      None        Level 0 - Base              govern
government      govern      -ment       Level 1 - Derivation        govern
governance      govern      -ance       Level 1 - Derivation        govern
>>>
```

Result:

All three words are normalized to govern, improving document clustering and topic modeling.

4. Morphological Parsing – Activate, Activation, Reactivation

Aim:

To identify prefixes, roots, suffixes, and derivational sequences.

Algorithm:

1. Start the program.
2. Initialize the words: activate, activation, and reactivation.
3. Read each word using a loop.
4. Check whether the word contains the prefix re-.
5. Identify the suffix transformation -ation.
6. Extract the common base word activate.
7. Determine the derivational sequence and meaning of each word.
8. Normalize all words to activate.
9. Display the prefix, root, suffix, derivational sequence, and normalized form.
10. Stop the program.

Python Code:

```
words = ["activate", "activation", "reactivation"]

for word in words:
    prefix = ""
    suffix = ""
    root = "activate"

    if word.startswith("re"):
        prefix = "re"

    if word.endswith("ation"):
```

```
    suffix = "ation"

if prefix and suffix:
    sequence = "activate -> activation -> reactivation"
    meaning = "Process of activating again"
elif suffix:
    sequence = "activate -> activation"
    meaning = "Process of activating"
else:
    sequence = "activate"
    meaning = "To make active"

print("Word:", word)
print("Prefix:", prefix or "None")
print("Root:", root)
print("Suffix:", suffix or "None")
print("Sequence:", sequence)
print("Meaning:", meaning)
print("Normalized:", root)
print()
```

Sample Output:

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/CO2 AT2 Q4.py
Word            Prefix    Root      Suffix          Normalized
activate        None      active    -ate            activate
Sequence: active -> activate
Word class: Adjective -> Verb
Meaning: To make active
------------------------------------------------------------------------
activation      None      active    -ate, -ion      activate
Sequence: active -> activate -> activation
Word class: Adjective -> Verb -> Noun
Meaning: The process of making active
------------------------------------------------------------------------
reactivation    re-       active    -ate, -ion      activate
Sequence: active -> activate -> reactivate -> reactivation
Word class: Adjective -> Verb -> Noun
Meaning: The process of activating again
------------------------------------------------------------------------
```

Result:

All three words are normalized to activate. The suffix changes the verb into a noun, while re- adds the meaning of repetition.

5. Inflectional Morphology – Create, Creates, Creating

Aim:

To identify grammatical suffixes and normalize word variants to their base form.

Algorithm:

1. Start the program.
2. Initialize the words: create, creates, and creating.
3. Read each word using a loop.
4. Check whether the word ends with -ing or -s.
5. If the suffix is -ing, remove it and restore the final e to obtain create.
6. If the suffix is -s, remove it to obtain create.
7. Otherwise, consider the word as the base form.
8. Classify each word as base form, third-person singular, or present participle.

9. Display the original word, suffix, grammatical category, root, and normalized form.
10. Stop the program.

Python code:

```
words = ["create", "creates", "creating"]


for word in words:
    if word.endswith("ing"):
        suffix = "ing"
        root = word[:-3] + "e"
        category = "Present Participle"

    elif word.endswith("s"):
        suffix = "s"
        root = word[:-1]
        category = "Third Person Singular"

    else:
        suffix = "None"
        root = word
        category = "Base Form"

    print("Word:", word)
    print("Suffix:", suffix)
```

```
    print("Category:", category)
    print("Root:", root)
    print("Normalized:", root)
    print()
```

Sample output:

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/CO2 AT2 Q5.py
Word           Suffix    Category                          Root        Normali
zed
create         None      Base form                         create      create
creates        -s        Third-person singular present     create      create
creating       -ing      Present participle                create      create
```

Result:

All words are normalized to create without changing their core meaning.